Prototypical Networks, Relation Network, and MAML

Step-by-Step Mathematical Example

Praveen Kumar Chandaliya

Department of Artificial Intelligence
Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

June 12, 2026

Prototypical Network

Step 1: Support Set

Consider a 3-way 2-shot classification problem.

Class A	**Class B**	**Class C**
$f(x_1^A) = [1, 2]$	$f(x_1^B) = [6, 5]$	$f(x_1^C) = [8, 1]$
$f(x_2^A) = [2, 1]$	$f(x_2^B) = [5, 6]$	$f(x_2^C) = [9, 2]$

These feature vectors are extracted by a CNN encoder.

Step 2: Compute Class Prototypes

The prototype of class k is

$$c_k = \frac{1}{N} \sum_{i=1}^{N} f_\theta(x_i)$$

Class A

$$c_A = \frac{[1,2] + [2,1]}{2} = [1.5, 1.5]$$

Class B

$$c_B = \frac{[6,5] + [5,6]}{2} = [5.5, 5.5]$$

Class C

$$c_C = \frac{[8,1] + [9,2]}{2} = [8.5, 1.5]$$

Step 3: Query Image

Suppose the query image is represented by

$$z_q = f_\theta(x_q) = [2, 2]$$

The query embedding will be compared with the class prototypes.

$$z_q = [2, 2]$$

Step 4: Euclidean Distance Computation

Distance between query and prototype:

$$d(z_q, c_k) = \|z_q - c_k\|^2$$

Distance to Class A

$$d_A = 0.5$$

Distance to Class B

$$d_B = 24.5$$

Distance to Class C

$$d_C = 42.5$$

Step 5: Compute Classification Probabilities

$$P(y = k|x_q) = \frac{\exp(-d_k)}{\sum_j \exp(-d_j)}$$

Using

$$d_A = 0.5, \qquad d_B = 24.5, \qquad d_C = 42.5$$

we obtain

$$P(A) \approx 1$$

$$P(B) \approx 0$$

$$P(C) \approx 0$$

Step 6: Final Prediction

Since

$$d_A < d_B < d_C$$

or equivalently

$$P(A) > P(B) > P(C)$$

The query sample is classified as

$$\boxed{\text{Class A}}$$

Relation Network

Step 1: Support Set

Consider a 3-way 1-shot classification problem. **Class A**

$$f_A = [1, 2]$$

Class B

$$f_B = [6, 5]$$

Class C

$$f_C = [8, 1]$$

Suppose the query image feature is

$$f_q = [2, 2]$$

Step 2: Feature Extraction

Both support and query images are passed through the same feature encoder:

$$f = \phi(x)$$

Obtained feature vectors:

$$f_A = [1, 2]$$

$$f_B = [6, 5]$$

$$f_C = [8, 1]$$

$$f_q = [2, 2]$$

Step 3: Construct Relation Pairs

Support and query features are concatenated. **Class A**

$$r_A = [1, 2, 2, 2]$$

Class B

$$r_B = [6, 5, 2, 2]$$

Class C

$$r_C = [8, 1, 2, 2]$$

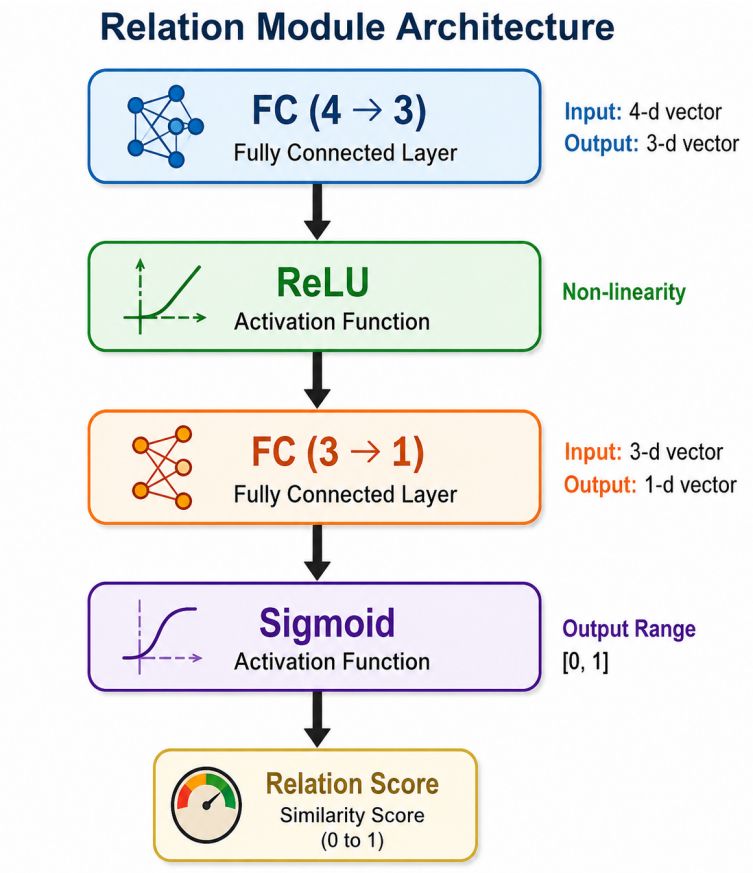
General formulation:

$$r = [f_s, f_q]$$

Step 4: Relation Module

The relation module learns similarity between support and query pairs.
Relation score:

$$s_i = g_\phi(r_i)$$



Step 5: Compute Relation Scores

Suppose the relation module outputs: **Class A**

$$s_A = 0.95$$

Class B

$$s_B = 0.12$$

Class C

$$s_C = 0.05$$

Scores lie in the range

$$0 \leq s_i \leq 1$$

Model-Agnostic Meta-Learning (MAML)

Step 1: Initialize Meta Parameters

Suppose the model contains one parameter:

$$\boxed{\theta = 5}$$

This parameter acts as a common initialization for all tasks.

Initial Meta Parameter

$$\theta = 5$$

Step 2: Sample Multiple Tasks

Consider three tasks:

- Task 1: Cat vs Dog
- Task 2: Car vs Truck
- Task 3: Bird vs Airplane

All tasks share the same initialization:

$$\theta = 5$$

Step 3: Task-Specific Adaptation

Task-specific parameters are obtained using

$$\phi_i = \theta - \alpha \nabla_\theta L_i(\theta)$$

Assume

$$\alpha = 0.1$$

Task 1

$$\nabla_\theta L_1 = 4$$

$$\phi_1 = 5 - 0.1(4) = 4.6$$

Task 2

$$\nabla_\theta L_2 = 2$$

$$\phi_2 = 5 - 0.1(2) = 4.8$$

Task 3

$$\nabla_\theta L_3 = 6$$

$$\phi_3 = 5 - 0.1(6) = 4.4$$

Step 4: Compute Query Losses

Suppose the adapted models produce:

$$L_1 = 0.20$$

$$L_2 = 0.10$$

$$L_3 = 0.15$$

Therefore, the meta-loss becomes

$$L_{\text{meta}} = L_1 + L_2 + L_3$$

$$L_{\text{meta}} = 0.20 + 0.10 + 0.15$$

$$\boxed{L_{\text{meta}} = 0.45}$$

Step 5: Compute Meta Gradient

Suppose

$$\nabla_\theta L_{\text{meta}} = 3$$

and the meta learning rate is

$$\beta = 0.01$$

The meta-gradient summarizes the losses from all tasks.

Step 6: Meta Parameter Update

Meta parameters are updated using

$$\theta \leftarrow \theta - \beta \nabla_{\theta} L_{\text{meta}}$$

Substituting values,

$$\theta_{\text{new}} = 5 - 0.01(3)$$

$$\boxed{\theta_{\text{new}} = 4.97}$$

Thus, the initialization improves after one episode.

Step 7: Repeat Multiple Episodes

Meta-learning is repeated over many episodes. Episode 1

$$\theta = 5$$

$$\downarrow$$

$$4.97$$

Episode 2

$$4.97$$

$$\downarrow$$

$$4.95$$

Eventually, the model learns a good initialization.

Step 8: Test Time Adaptation

Suppose a new task arrives:

Horse vs Zebra

with only a few training samples. Start from

$$\theta = 4.8$$

Suppose

$$\nabla_\theta L = 2$$

Using

$$\alpha = 0.1$$

the new task parameter becomes

$$\phi_{\text{new}} = 4.8 - 0.1(2)$$

$$\boxed{\phi_{\text{new}} = 4.6}$$

Step 9: Final Prediction

The learned meta parameters allow fast adaptation.

$$\boxed{\theta \rightarrow \text{Fine-Tune} \rightarrow \phi_{\text{new}} \rightarrow \text{Prediction}}$$

MAML learns how to learn new tasks using only a few examples.